



# Spécifications techniques

|                           |          |
|---------------------------|----------|
| <b>Stack Technique</b>    | <b>2</b> |
| <b>Architecture Java</b>  | <b>2</b> |
| <b>Gestion des images</b> | <b>2</b> |
| <b>Sécurisation</b>       | <b>2</b> |
| <b>Outils</b>             | <b>3</b> |

## Stack Technique

Pour ce projet, utiliser **les versions suivantes ou des versions plus récentes** :

- **Angular 20**
- **Java 11 ou 17**
- **Spring Boot**
- **Spring Security avec authentification via JWT**
- **Base de données MySQL**
- **Code versionné sur un repo Git**

**Attention** : Il ne faut pas modifier le front-end de l'application.

## Architecture Java

L'application est développée avec une **architecture en couche (Controller/Service/JPA Repository)**.

## Gestion des images

Lors de la création d'une location, une image est requise. Elle sera envoyée à l'API pour être stockée sur le serveur, puis l'URL de cette image sera enregistré en base dans la table location.

## Sécurisation

En utilisant Spring Security et JWT, suivre les bonnes pratiques : toutes les routes nécessitent l'authentification (sauf celle de création de compte, du login et de la documentation Swagger).

S'assurer que le mot de passe est crypté en base de données et que les identifiants de la base de données n'apparaissent pas dans le code.

**Attention** : Bien que la documentation OpenAPI (Swagger) soit accessible sans authentification, il faudra s'identifier pour pouvoir tester les appels d'API authentifiées.

## Outils

**Mockoon** : cette application permet de simuler les réponses d'un serveur.

- Il faudra créer toutes les routes présentes dans Mockoon.
- Notez que pour chaque route, il existe plusieurs réponses (par exemple une réponse OK avec un status 200, et une réponse unauthorized avec un status 401).

**Swagger** : cette application permet de documenter toutes les routes de l'API en suivant la spécification OpenAPI.